Big Data Infrastructure Technologies for Data Analytics (BDIT4DA)
Agentic AI Systems and Systems and Software Engineering (SSE)

BDIT4DA Project Development Guidelines

Prompt Engineering and Practical Examples

Yuri Demchenko
IvI UvA, NTUU “Igor Sikorsky KPI”

Outline

Part 1 - Introduction

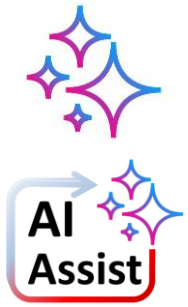
- Agentic AI and Agentic Systems: Agents, Models, Architecture, Components
- Prompt Engineering (PE)
  - PE core techniques and
  - Foundational prompt construction patterns

Part 2 - Examples

- Prompt example for Project development stages

General

- Background information for LLM and Agentic systems: Knowledge Complexity Model (KCM) and Human-AI/LLM interface
- Required knowledge for efficient GenAI assisted project development
  - Foundational architecture design principles
  - Project/Course/domain specific concepts and principles



Sign indicating Generated with GenAI/ChatGPT

or Assisted with ChatGPT (GenAI)

Disclaimer

This research and development is based on and built around the concept of Knowledge.

- Knowledge doesn't have measurable properties similar to (research) experiments.
- Knowledge may have a variety of presentation and expression forms
- However, knowledge can be validated with epistemic correctness – attributed only to humans/researchers and dependent on their personal epistemic model

Consequently, the presented information (working knowledge) may have variable form; however, it clearly represents the author's epistemic model of the research topic.

Agentic (AI) Systems Design – From Components to Actions & GenAI/LLM-assisted Systems Design

- Generative AI (GenAI) – General Purpose AI – Artificial General Intelligence
  - GenAI - AI models specifically trained to generate new content, such as text, images, code, audio, or video, based on learned patterns from massive datasets; the perceptual and creative engine behind modern AI systems (e.g., LLMs, diffusion models)
- Agentic (AI) systems extend architecture design from specifying components, services, interfaces, and control flows to specifying how execution plans may be constructed and revised at runtime.
  - Agentic AI Systems / Agentic AI / Agentic Systems: AI architectures designed to operate with agency, meaning they can autonomously plan, reason, use tools, iterate on intermediate steps, and execute multi-step workflows to achieve a high-level goal defined by a human
  - Usage: Agentic AI Systems – formal technical and academic; Agentic AI – referring to the property of agency (contrasting static AI with agentic behavior, workflow/tactic optimisation); Agentic systems – general (self-managed) agents, infrastructure, task/goal focused.
- Architects must define the objectives, knowledge, capabilities, authority boundaries, invariants, and verification mechanisms within which this adaptation occurs.
- Runtime engineering makes those boundaries operational and establishes evidence that the resulting actions achieve the intended outcome.
- GenAI/LLM-assisted Systems Design – Efficient Prompt Engineering
  - Prompt should provide project context, expected output and goal, gradual staged information + design principles and additionally guardrails

Background - Recommended Project Development Steps – Refer to the Project Guidelines

BDIT4DA group project development steps

- **Step 1 - ADDE.** Describe your imaginary company that you want to transform to the ADDE (Agile Data Driven Enterprise) type (you can also use your employment company as an example). Select the industry sector and your particular business activity.
- **Step 2 - WebPresence.** The company should have a web presence and you need to decide what services you provide on the web, what data you use and what data you can collect from the web. Remember that one of data types or sources used by ADDE is social media data. Provide suggestions on how you can use social media data for your company.
- **Step 3 - DataFlow.** Describe **data flows** in your company. Identify where you will use data collected from the web and other sources of information like sell statistics, user activity, or social media activity.
- **Step 4 - DataAnalytics.** Suggest what **data analytics methods** you will use. Search for similar use cases and solutions they use. You need to develop and demonstrate the prototype of the working application.
- **Step 5 - BDplatform.** Select a suitable **platform** for general data management and a Big Data Analytics platform for that can implement the data flow and data analytics processes.
- **Step 6 – DevDemo: Development and implementation.** This step is focused on the implementation of data analysis methods and preparation of a demonstration of a working algorithm. Full deployment of the algorithm on a cloud platform is not required.
- **Step 7 - DMG.** Define a **data management policy**, including data quality, organisational data governance architecture, and procedures.
- **Step 8 - BDSecurity.** Address **data protection and access control**, and **compliance** issues.

Recommended Project Development Steps – Stages in working with AI/Chatbots and Project development

Stage 1 – Mandatory preparation for effective use of LLM/chatbots

- Context of the project development and preparation for design stage
- Familiarity with the course foundational aspects and architecture aspects

- Step 1 - ADDE
- Step 2 - WebPresence
- Step 3 - DataFlow

Stage 2 – Design stage: Concise iterative (human governed) work with LLM/chatbots – Generative – Productive – Learning

- Beneficial knowledge of the relevant standards, optional references to design best practices – as prompt context

- Step 4 - DataAnalytics
- Step 5 - BDplatform

Stage 3 – Development

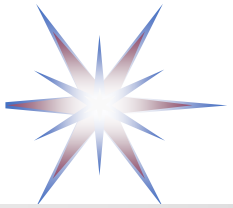
- Agentic coding is not forbidden for initial version, reported code must demonstrate critical review and understanding

- Step 6 – DevDemo: Development and implementation

Stage 4 – Project finalisation and non-functional aspects

- Guidelines and templates are provided and supported by practical/lab sessions

- Step 7 - DMG. Define a data management policy, including data quality, data governance, and procedures.
- Step 8 - BDSecurity. Address data protection and access control, and compliance issues.



Prompt Engineering

- What is a prompt?
- How is a prompt used in LLM/Model/Chatbot?
- Prompt as a mechanism to provide context in Human-AI interaction

Human - AI Epistemic Interaction Model for achieving goal/intention and validated output

Human agent:

Personal epistemic model
- semantic memory
- methodological knowledge
- tacit expertise
- epistemic judgment
- goals / intentions
- **prompt / action**

AI agent:

Agent knowledge state
- **context window (prompt + config context)**
- **retrieved knowledge**
- latent model representation (ML internal)
- symbolic/graph memory
- tool outputs
- generated presentations

Interaction process:

Human intention
↓
Prompt / instruction presentation
↓
AI internal **representation**
↓
Generated knowledge presentation (output)
↓
Human perception and reconstruction/representation
↓
KCM/KCEG assessment (epistemic judgement by human/researcher)
↓ ↑ ↓ ↑
HGLEP-guided refinement (iterative)
↓
Verified/approved **result/output**
↓
Updated human and AI knowledge states

KCM – Knowledge Complexity Model
KCEG – Knowledge Complexity and Epistemic Governance
HGLEP – Human – Governed LLM Epistemic Knowledge Production

Knowledge Presentation – Representation - Perception

Available/Received Knowledge

Knowledge Presentation

1. Vocabulary/Terminology complexity (KC1)
2. Semantic complexity (KC2)
3. Ontological complexity (KC3)
4. Structural complexity (KC4)
5. Methodological complexity (KC5)
6. Epistemic complexity/validity (KC6)

Knowledge inferential/LLM, Internal representation

Knowledge Representation

1. Vocabulary / terminology (K1)
2. Syntax / expression form (K2)
3. Factual / propositional layer (K3)
4. Semantic layer (K4)
5. Ontological layer (K5)
6. Structural / graph layer (K6)
7. Inferential layer (K7)
8. Methodological layer (K8)
9. Epistemic / research-frontier layer (K9)

Perception via Information, Epistemic Validation, Internalisation

Knowledge perception

- If a human doesn't have a full Kx or KCx model of KCM, they may treat received/observed information structured wrongly or a kind of imaginary mixed
- This is the case for children
  - Can we control and speed up development?
  - Can we do the same for Doctoral (PhD) Students?
- Children imaginary treat objects and animals with human features
  - Hypothetically, because of their developing perception KCM
  - Agent-centered perception model by children
- TODO: Adding Pragmatics to Semantics Complexity level as related to language understanding by humans

KCM – Knowledge Complexity Model
KCEG – Knowledge Complexity and Epistemic Governance
HGLEP – Human – Governed LLM Epistemic Knowledge Production

Core Prompt Engineering (PE) Techniques

Technique	Mechanism	Application to agentic development and operation	Important qualification
Direct instruction and contextual prompting — "simple context"	Provide a task, relevant background, constraints, and expected output.	Supply requirements, architecture boundaries, repository conventions, or an operational incident description.	"Simple context" is an informal description. Context quality and relevance matter more than volume.
Zero-shot prompting	Describe the task without demonstrations.	Routine classification, extraction, summarisation, or a clearly specified coding task.	Zero-shot means no examples; it does not mean no instructions or context.
One-shot / few-shot prompting	Include one or several demonstrations (solution example) of the desired input–output behaviour.	Demonstrate an API convention, acceptable patch, incident classification, or architecture decision record.	Examples should represent important variations and edge cases. In-context examples ordinarily do not update model weights.
Role / persona prompting	Assign a professional perspective, responsibility, or communication style.	Review this change as a reliability engineer, checking failure recovery and observability.	A role label does not confer expertise or authority. Explicit review criteria make the role operational.

Foundational prompt-construction patterns

Technique	Mechanism	Application to agentic development and operation	Important qualification
Chain-of-Thought — CoT	Elicit intermediate reasoning; the original approach supplies worked reasoning examples.	Analyse dependencies, diagnose faults, or work through a constrained design problem.	A plausible explanation is not proof of correctness or necessarily a faithful account of internal computation. Wei et al., 2022.
Zero-shot CoT	Request stepwise reasoning without worked examples; the classic cue is “Let’s think step by step.”	Establish a reasoning baseline when suitable demonstrations are unavailable.	The original improvements were model- and benchmark-dependent; the phrase is not a universal performance improvement. Kojima et al., 2022.
Tree of Thoughts — ToT	Generate alternative intermediate solution states, evaluate them, and search through promising branches, including backtracking.	Explore competing designs or repair strategies where early decisions affect later possibilities.	The original method includes a search procedure. Merely requesting “three alternatives” does not implement full ToT. Yao et al., 2023.
Meta-prompting	Use higher-order instructions to construct prompts, organise subtasks, or coordinate specialised model calls.	Generate task-specific agent instructions or orchestrate specialist reviews.	The term has multiple meanings in the literature; specify which mechanism you mean.
ReAct (Reasoning + Action)	Interleave reasoning, actions, and observations from tools or the environment.	Inspect code → choose an edit → run tests → inspect failures → revise.	It is an interaction pattern supported by an execution loop. Actual tool results must inform subsequent decisions.
Agent harness (environment & tools)	Runtime software manages model calls, tools, context, state, execution limits, and recovery.	Sustain a coding or operations agent across many steps and sessions.	A harness is not a prompting technique. It hosts and operationalises prompting and agent workflows.

Foundational sources

- Foundational sources (TODO refs)
  - Wei et al. for CoT,
  - Kojima et al. for zero-shot CoT,
  - Yao et al. for ToT and ReAct,
  - Distinct meta-prompting formulations of Suzgun and Kalai and Zhang et al.
  - Harness design is documented particularly clearly in Anthropic’s engineering accounts of long-running agents.

Structured output versus structured reasoning – For development and operation

- Requesting output form: JSON, a decision table, or a fixed report schema to structure the model's *output*, is not sufficient
  - It does not establish the correctness of the underlying reasoning
- Where software consumes the result, ***pair*** the output specification with schema validation and domain checks
  - Likewise, a requested plan should be treated as an inspectable work product, not as evidence that the planned actions occurred

Structured output vs/and Structured reasoning

Situation	Useful starting combination	Evidence of success (validation/judgement)
A bounded task with clear requirements	Direct instructions + relevant context + output specification	Valid output and task-specific acceptance checks
Repeated tasks with project conventions	Few-shot examples + reusable skills	Consistency across representative and exceptional cases
Architecture analysis	Explicit constraints + decomposition + comparison of alternatives	Traceable trade-offs, interface consistency, and reviewable decisions
Coding and debugging	ReAct + repository retrieval + tool execution	Correct changes supported by meaningful tests
Operational diagnosis	ReAct + current observations + explicit action boundaries	Verified system state and recorded action outcomes
Long-running development	Persistent state + context management + harness + verification checkpoints	Recoverable progress and demonstrated completion
Improving agent performance	Versioned prompts + evaluation-driven optimisation	Better results on held-out tasks, with measured cost and latency

PE Techniques for Different Project Development Stages

Step 1, Step 2

Step 3

Step 4

Step 5

Step 6

Project Development Steps

Project stage	Recommended techniques	Practical application and expected output
1. Problem framing and objectives	Contextual prompting; clarification questions; stakeholder-role prompting	Establish the problem, stakeholders, scope, constraints, assumptions, and success criteria. Output: agreed project brief.
2. Requirements and acceptance criteria	Structured prompting; few-shot examples; decomposition; critique against a checklist	Develop functional requirements, quality requirements, scenarios, and testable acceptance criteria. Output: requirements specification with traceability.
3. Architecture exploration and selection, validation	Alternative generation; role-based reviews; structured comparison; selective Tree-of-Thoughts exploration	Compare plausible architectures against requirements and constraints. Output: architecture views, interface boundaries, and architecture decision records.
4. Detailed design and work planning	Task decomposition; least-to-most prompting; prompt chaining; few-shot design templates	Define component responsibilities, data models, interfaces, dependencies, and implementation tasks. Output: component specifications and a dependency-aware plan.
5. Implementation and integration	Contextual prompting grounded in the repository; few-shot conventions; bounded ReAct; tool-assisted iteration	Implement a defined change, inspect relevant code, compile, test, and correct failures. Output: reviewable changes with verification evidence.
6. Verification and validation	Criteria-based review; adversarial test-case generation; evaluator–optimizer workflows; evidence-driven refinement	Check correctness, integration, quality requirements, and fitness for the intended use. Output: test results, defect records, and acceptance assessment.
7. Deployment and handover	Checklist prompting; structured output; grounded prompt chaining; constrained tool use	Prepare configuration, deployment steps, operational documentation, rollback procedures, and smoke tests. Output: reproducible release and handover package.
8. Operation, maintenance, and evolution	Retrieval-grounded prompting; ReAct for diagnosis; hypothesis comparison; reflective learning from verified outcomes	Analyse observations, diagnose incidents, assess change impact, and propose improvements. Output: incident or change record with evidence.

Common Prompt Structure

Prompt element	What to specify
Objective	The engineering task and intended outcome
Authoritative context	Approved requirements, architecture decisions, interfaces, repository material, or observations – *Provided as additional documents (prompt context)*
Scope and constraints	What may change, what must remain consistent, and relevant quality or resource limits
Working method	Clarification, alternatives, decomposition, tool use, or review appropriate to the stage
Expected artifact	A requirements table, architecture comparison, patch, test report, or deployment procedure
Acceptance evidence	How completion and correctness will be demonstrated
Uncertainty handling	Which missing facts require clarification and how assumptions must be recorded

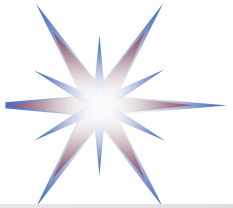
Template is important => Not all elements may be present => It may develop and be extended as the project progresses
=> Prompt is provided in each request & Context may be provided as external documents, which can also evolve

Selecting PE Techniques- When to use

- **Simple Context** or **Direct Instruction and Contextual Prompting:** Main and initial techinique
- **Chain-of-thought prompting:** request useful engineering artifacts - assumptions, calculations, trade-offs, and concise justification. A long reasoning narrative is not validation.
- **Role prompting:** attach concrete responsibilities and review criteria to the role. “Act as an architect” is insufficient guidance.
- **Tree of Thoughts:** reserve explicit branching search for decisions that warrant the added effort. A structured comparison of a few alternatives is often sufficient.
- **Self-refinement:** connect revision to tests, specifications, or external evidence. Self-critique alone can preserve the original mistake.
- **ReAct:** use when interaction with tools or an environment is necessary, with bounded scope and observable results.
- **Meta-prompting:** introduce after a repeatable task and evaluation criteria exist; it can help improve stage-specific prompt templates.
- **Harnesses:** treat these as runtime engineering. They become relevant when an agent performs sustained actions and must maintain state, recover, and respect execution boundaries.

Foundation Principles in Architecture Design

- Foundational principle - How it improves prompting
  - Separation of concerns Defines distinct responsibilities and prevents an underspecified request from combining unrelated functions.
- Abstraction and encapsulation
  - Specifies required behaviour and exposed contracts without prescribing every implementation detail.
- Modularity, cohesion, and coupling
  - Defines component boundaries and acceptable dependencies.
- **Layering**
  - Specifies responsibilities for each layer and permitted dependency directions
  - Interfaces and interoperability
  - Makes **data structures**, semantics, errors, and interaction expectations explicit
- Quality attributes and trade-offs
  - Converts broad requests such as “robust” or “scalable” into concrete conditions and priorities.
- State, concurrency, and failure management
  - Includes conditions that a normal-path functional description may omit.
- Traceability and architectural decisions
  - Connects generated artifacts to requirements and previously accepted decisions.
- Verifiability and observability
  - Requires testable behaviour and evidence of execution.
- Common Information Model (CIM) ensuring consistency in IDs of elements, processes and variables across the whole system and design process
  - Applied across all architecture design principles



Case Study – BDIT4DA project development

- Steps from the Project Guidelines
- Preparation – Steps 1-3
- Prompt design for Project Step 4 and Step 5
  - Prerequisite Steps 1 -3 fulfilled and Big Data Architecture Framework understood
- Prompt examples

- Note: All Prompt examples use an educational FreshRoute company serving grocery logistics and delivery for regional customers

Project Development Stages and Architecture Focus

Project milestone <= Focus and Outcome

- Problem and requirements
  - “What must the system achieve, and under which conditions?”
- Dataflow
  - “What data are present, how they represent business processes, what are sources and types of data?”
- Architecture
  - “How are responsibilities allocated, and why are these boundaries appropriate?”
- Prompt preparation
  - “Which architectural decisions and constraints must the AI preserve?”
- Implementation review
  - “Does the generated work conform to those decisions?”
- Verification
  - “What evidence supports the claimed behaviour and quality?”
- Change and evolution
  - “What can change independently, and what must be revalidated?”

Recommended Project Development Steps – Refer to the Project Guidelines

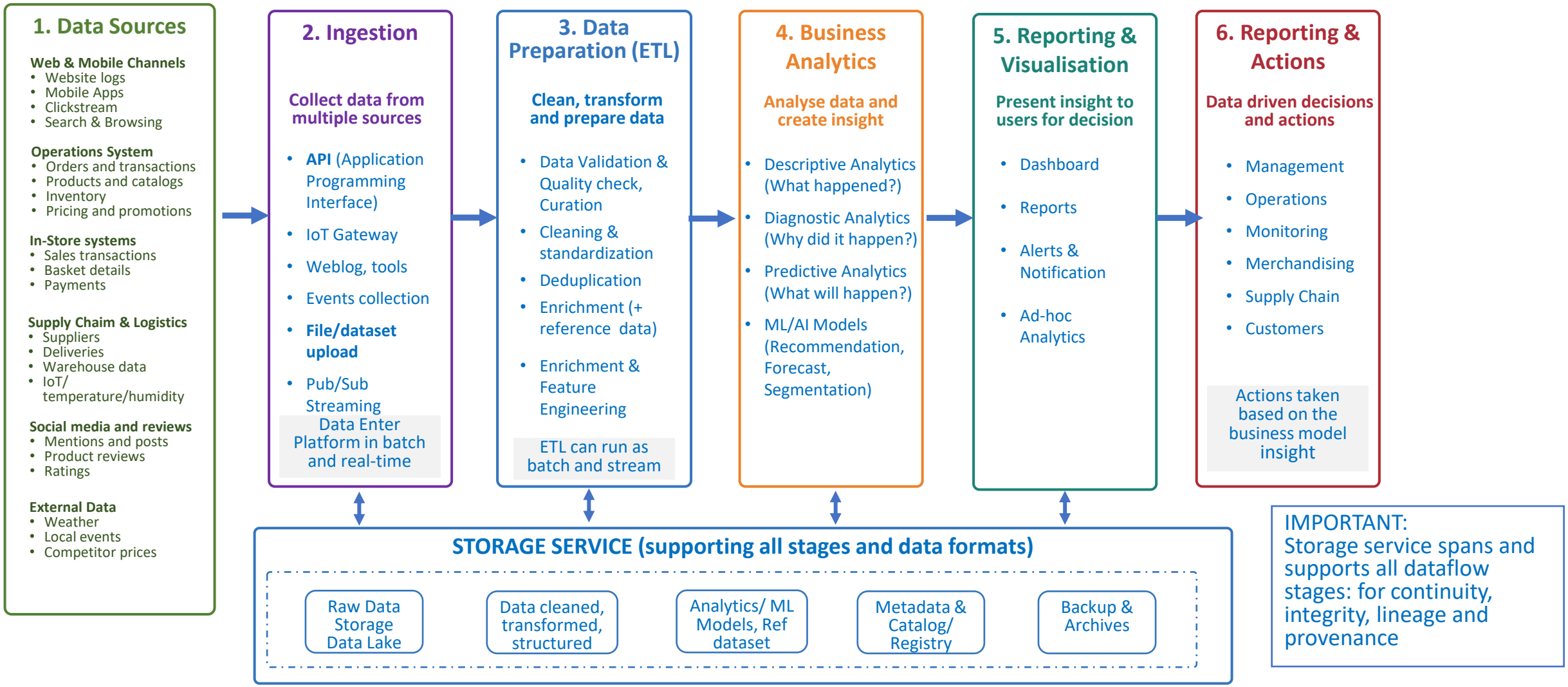
BDIT4DA group project development steps

- **Step 1 - ADDE.** Describe your imaginary company that you want to transform to the ADDE (Agile Data Driven Enterprise) type (you can also use your employment company as an example). Select the industry sector and your particular business activity.
- **Step 2 - WebPresence.** The company should have a web presence and you need to decide what services you provide on the web, what data you use and what data you can collect from the web. Remember that one of data types or sources used by ADDE is social media data. Provide suggestions on how you can use social media data for your company.
- **Step 3 - DataFlow.** Describe **data flows** in your company. Identify where you will use data collected from the web and other sources of information like sell statistics, user activity, or social media activity.
- **Step 4 - DataAnalytics.** Suggest what **data analytics methods** you will use. Search for similar use cases and solutions they use. You need to develop and demonstrate the prototype of the working application.
- **Step 5 - BDplatform.** Select a suitable **platform** for general data management and a Big Data Analytics platform for that can implement the data flow and data analytics processes.
- **Step 6 – DevDemo: Development and implementation.** This step is focused on the implementation of data analysis methods and preparation of a demonstration of a working algorithm. Full deployment of the algorithm on a cloud platform is not required.
- **Step 7 - DMG.** Define a **data management policy**, including data quality, organisational data governance architecture, and procedures.
- **Step 8 - BDSecurity.** Address **data protection and access control**, and **compliance** issues.

BDIT4DA Project Practice Preparation – For efficient prompt design

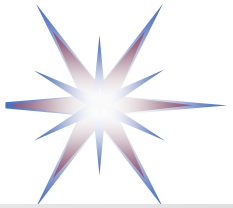
- In the course "Big Data Infrastructure Technologies for Data Analytics", students need to develop an architecture, dataflow, data analytics services, and implementation on one of the cloud platforms (typically AWS, Azure, GCP, others) for their hypothetical Agile Data Driven Enterprise (typically SME - Small to Medium Enterprise) .
- Before starting to actively use Agentic AI and prompting, the student should already have an initial version of the following:
  - General project description, business model, services, used/processed data/datasets, initial understanding of necessary analytics and ML model to be developed and operated.
- **What kind of contextual information should the students provide for effective prompting (project outcome focused)**
  - Project guidelines – sections describing required outcome (project Steps or stages)
  - Big Data architecture related - course materials, general architecture design principles – document or formal reference
- Recommended prompt techniques
  - simple contextual prompting,
  - structured prompting,
  - one/many-shots prompting, and
  - role prompting where beneficial
- Necessary knowledge:
  - Foundational knowledge of the Big Data architecture and functional components
  - General architecture design principles and software architecture

FreshRoute Dataflow – Summary Results Step 3



Selecting Prompt Techniques for BDIT4DA Project Development

- Suggested prompting for a few initial project development stages:
  - 1. Problem framing and objectives
    Contextual prompting; clarification questions; stakeholder-role prompting
  - 2. Requirements and acceptance criteria
    Structured prompting; few-shot examples; decomposition; critique against a checklist
  - 3. Architecture exploration and selection
    Structured prompting; alternative generation; role-based reviews; structured comparison; selective Tree-of-Thoughts exploration
  - 4. Detailed design and work planning
    Task decomposition; least-to-most prompting; prompt chaining; few-shot design templates
  - 5. Implementation and integration
    Contextual prompting grounded in the repository; few-shot conventions; bounded ReAct; tool-assisted iteration
- Contextual information to be shared and extended along project design stages
  - Included as context into the next step or stage prompting



Prompt Examples

- Project context – Example and template – Educational Project FreshRoute
- PE01. Simple contextual prompting
- PE02. Structured prompting
- PE03. Role prompting – Useful for Draft Review
- PE04. One-shot and few-shot prompting

- Note: All Prompt examples use an educational FreshRoot company seving grocery delivery for regional customers

Preparation – Project Context (Example/Template) – Provided as part of the prompt

PROJECT CONTEXT — FRESHROUTE

We are developing FreshRoute for the course Big Data Infrastructure and Technologies for Data Analytics. Steps 1–3 are completed in the attached report. We are now working on Step 4 — Data Analytics.

FreshRoute provides online grocery ordering, home delivery, subscription baskets, delivery tracking, and personalised offers. Its business problems include fluctuating demand, stockouts, food waste, and delivery delays.

The proposed data sources are:

Orders and sales: structured operational SQL data.

Web clickstream: JSON events.

Driver GPS and delivery events: streaming location and operational events.

Social comments and reviews: unstructured text.

Weather: periodically refreshed external data.

The existing dataflows connect:

Orders and weather to demand forecasting and inventory planning.

Clickstream to customer behaviour analysis and marketing.

GPS and delivery events to ETA prediction and delivery monitoring.

Social text to sentiment analysis and issue reporting.

Raw heterogeneous data land in appropriate operational or raw stores.

Processing produces curated datasets, features, and aggregates for analytics and warehouse reporting.

The report's figures, including 25,000 daily users and 6,000 daily orders, are hypothetical design estimates, not measurements.

Actual datasets or samples available to our group:

[Specify files, fields, time coverage, and access conditions; otherwise write “not yet available”.]

Prototype constraints:

[Specify available time, skills, execution environment, and any budget limit.]

Use the completed report as the project baseline. Distinguish documented facts, design estimates, assumptions, and new proposals

PE01. Simple contextual prompting

PE01. Simple contextual prompting uses ordinary prose to request a coherent first proposal. “Simple” means a straightforward instruction, not an absence of relevant context.

PROMPT

> Using the FreshRoute context above and the attached Steps 1–3 report, help us develop Step 4 — Data Analytics.
>
> Recommend suitable analytics methods for the business needs already identified. Explain which data each method needs, what processing is required, what output it produces, and who uses that output to make a decision. Include simple analytical or statistical approaches where these are sufficient; do not assume every task requires ML.
>
> Recommend one manageable working prototype for our group. Explain its input, method, output, place in the existing dataflow, and how we could test it. Identify missing data or labels and distinguish a proposed experiment from a demonstrated result.
>
> Find two relevant, verifiable published use cases or technical studies, and explain what is transferable to FreshRoute and what differs. If browsing is unavailable, provide search queries and source-selection criteria instead of inventing references.
>
> Keep detailed cloud-provider selection for Step 5. Finish with the questions we must resolve before implementing the prototype.

PE01-S. Simple contextual prompting – Activating Structured inference and output

PE01-S. Simple contextual prompting with textual description of the expected output structure and generated candidate knowledge complexity

PROMPT Example of the multistep iterative process – Instructing chatbot to correct/redesign provided diagram in the step N-1

Step N

Returning to your initial answer in this chat (see attached doc), do map the suggested components into a layered architecture diagram (e.g., physical layer → data layer → twin/simulation layer → monitoring/decision layer → user layer), so it's easier to use in the framework description

Step N+1

Your diagram is quite informative and actually useful for EIMS components development. But for a proposal or a paper diagram, it is not visual.

Redesign it in the following way: (1) Layers should be stacked on top of each other, with Layer 1 at the bottom and Layer 5 on top; (2) components of the same layer should be placed either inside block representing layer, or just listed inside block; (3) links between modules should not be individual but should be just shown as links between layers; (4) Security as cross-layers service should just shown as Security plane left or right of stacked layers.

Use the above conventions.

PE02. Structured prompting

2. Structured prompting: This version makes the engineering relationships and expected evidence explicit. It is suitable for producing the main Step 4 report draft.

PROMPT
TASK
Develop a Step 4 Data Analytics proposal for FreshRoute using the shared project context and attached documents.
PROJECT BOUNDARY
Develop the analytical responsibilities already identified in Steps 1–3. Treat weather-informed demand forecasting as an extension of demand forecasting unless there is a reason to separate them. Distinguish ETA prediction from route optimisation: predicting arrival time does not itself determine an optimal route.

ARCHITECTURAL CONSTRAINTS
Preserve the separation of ingestion, storage, data preparation, analytics/model execution, and presentation.
Define the inputs and outputs between these responsibilities.
Explain how raw events or text become analytical features or warehouse-ready outputs.
Do not introduce streaming computation solely because the source arrives as a stream; justify the required output freshness.
Keep the design logically specified so it can be mapped to a cloud platform in Step 5.

OUTPUT A — ANALYTICS INVENTORY
Provide one record per analytical use case:
Business question and intended user.
Decision or action supported.
Analytics type: descriptive, diagnostic, predictive, or prescriptive.
Required sources and fields, marked available or missing.
Unit of analysis and relevant time window.
Preparation, joins, and data-quality checks.
Simple baseline and one justified candidate method.
Output and its interface with the existing dataflow.
Batch or streaming execution and justified freshness.
Evaluation method and principal limitations.

OUTPUT B — PROTOTYPE SELECTION
Compare demand forecasting, clickstream analysis, ETA prediction, and sentiment analysis against business relevance, data availability, implementation effort, and verifiability. Select one prototype and justify the choice. Do not assume the most sophisticated method is the best choice.

OUTPUT C — PROTOTYPE SPECIFICATION
Define:
A bounded business question.
Minimum input schema and data coverage.
Data preparation and analytical method.
Output schema and a small dashboard or other usable presentation.
The distinction between training and inference, where applicable.
A baseline, evaluation data, metrics, and acceptance criteria.
Handling of missing inputs, invalid records, and insufficient history.
An implementation sequence and required demonstration evidence.
For forecasting, use a time-aware evaluation design and consider whether inputs would actually be known at prediction time. Explain any limitations in using observed sales as a measure of demand.

OUTPUT D — RELATED WORK
Find two verifiable published use cases or technical studies. For each, provide a source link, task, data, method, reported evaluation, and applicability to FreshRoute. Do not transfer another project's performance claims to our project. If browsing is unavailable, give a search plan.

OUTPUT E — OPEN ISSUES
List assumptions, missing evidence, and decisions requiring group approval. Do not claim that data have been analysed or a prototype has run unless execution evidence is supplied.

PE03. Role prompting – Useful for Draft Review

3. Role prompting: Role prompting is particularly useful **after** a draft exists (Step 4). It gives the review a defined responsibility rather than simply requesting another solution.

PROMPT
Act as a data analytics architect reviewing a student team's FreshRoute Step 4 proposal.
You have the attached project guidelines, the completed Steps 1–3 report, our shared project context, and the Step 4 draft pasted below. Your responsibility is to assess consistency, feasibility, and verifiability.

Review the proposal against these criteria:
Every analytical task supports an identified business question and user.
Required datasets, fields, labels, identifiers, and time coverage are available or explicitly identified as gaps.

Ingestion, preparation, analytics, and presentation have clear responsibilities.
Analytical outputs fit the existing dataflow and have usable interfaces.
Batch or streaming choices follow freshness requirements.
Method complexity is justified against a simpler baseline.
Evaluation avoids inappropriate use of future information and measures the stated task.
Failure handling and prototype completion evidence are adequate.

Return:
A short overall assessment.
A review table with issue, evidence from the draft, engineering consequence, and recommended correction.
A distinction between blocking issues, important improvements, and optional extensions.
Five questions that the student team should be able to answer orally.
A recommendation: ready to implement, ready with stated conditions, or requiring revision.
Do not rewrite the entire project or introduce new requirements without identifying them as proposals. Do not treat a polished explanation as evidence of a working implementation.
STEP 4 DRAFT
[Attached - Paste the team's proposal.]

PE04. One-shot and few-shot prompting

4. Preparing one-shot and few-shot prompting

- “Shot” is a worked input–output example of the analytical specification students should produce (Step 4). The project description itself is context, not a shot.
- First stage-shot: Use descriptive sales analytics as the first example, then specify demand forecasting. This keeps the structure while recognising that prediction introduces additional requirements.

WORKED EXAMPLE — DAILY SALES ANALYTICS

WORKED EXAMPLE — DAILY SALES ANALYTICS

Example input

Business question: Which product categories account for the most completed sales each day?
User: Operations manager.
Decision supported: Identify sales patterns for subsequent inventory review.
Proposed data: order_id, order_timestamp, product_id, quantity, net_line_amount, order_status; product_id and category in a product table.
Assumptions: These fields can be obtained; currency and business timezone are consistent; each order line can be identified uniquely.

Example output

Analytics type: Descriptive.
Unit of analysis: Product category per business day.
Preparation: Validate identifiers and amounts, resolve duplicate order lines, apply a documented treatment of cancellations and refunds, and join products to categories.
Method: Group and aggregate net sales and quantities using SQL or equivalent dataframe operations. ML is unnecessary.
Logical dataflow: Operational order and product data → validated analytical tables → daily category aggregates → dashboard and warehouse reporting.
Output contract: business_date, category, net_sales, units_sold, with documented currency, units, and calculation rules.
Execution: Daily batch, provided that next-day reporting satisfies the user's need.
Verification: Use a small fixture with manually calculated expected totals; check category totals reconcile with included source records.
Failure handling: Report unmatched products and invalid records; flag incomplete input rather than silently reporting it as complete.
Limitation: Recorded sales do not measure all customer demand, because unavailable products may prevent purchases.

Using One-shot technique

INPUT: The worked daily sales analytics example as a model for the structure and precision of your answer, specify a FreshRoute demand-forecasting prototype.

PROMPT

Use the Steps 1–3 report and our actual data inventory. Propose a manageable forecasting unit, horizon, and update frequency, and label these as decisions requiring approval.

Adapt the example's fields to forecasting. Include the prediction target, historical coverage, baseline, candidate method, time-based evaluation, and availability of features at forecast time. Explain whether weather data can be used without introducing information unavailable at prediction time.

Preserve the logical separation between data preparation, model training, prediction generation, and presentation. State what changes are needed if the available data cannot support the proposed task.

Do not copy the descriptive example's aggregation method or acceptance test into the forecasting task without justification.

Few-shot prompting - Prepare two additional checked examples with the same specification structure

Example	What it should demonstrate
Review sentiment classification	Text preparation, labelled evaluation samples, a simple baseline, classification errors, and aggregation of results for reporting.
Delivery delay monitoring	Event identifiers and timestamps, justified freshness, late or duplicate events, and operational alerts; clearly distinguish monitoring from ETA prediction.

What students should compare

- Evaluate all outputs against the same five questions:
- Does the proposed analytics support an identifiable business decision?
- Can the required data actually support the task?
- Are responsibilities, dataflow, and interfaces coherent?
- Are method complexity and execution frequency justified?
- Is there a credible way to demonstrate correctness and usefulness?

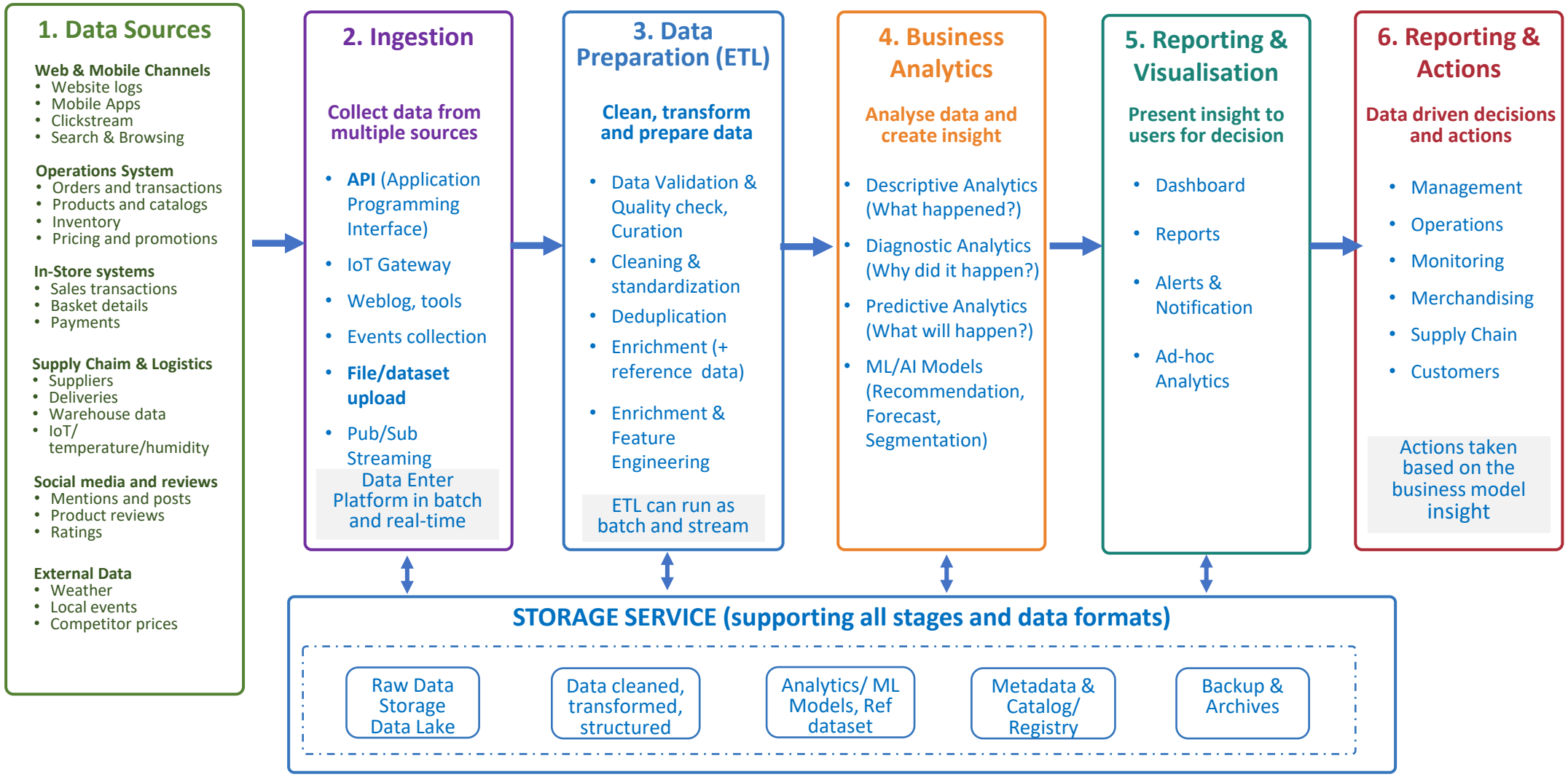
Preparation should follow this sequence:

- Students draft the additional examples using their course knowledge.
- The teacher or peers check technical correctness, assumptions, and consistency with Steps 1–3.
- Use one example first, then the three approved examples.
- Keep the target task and project context unchanged.
- Compare whether additional examples improve completeness and transfer—or encourage inappropriate copying.

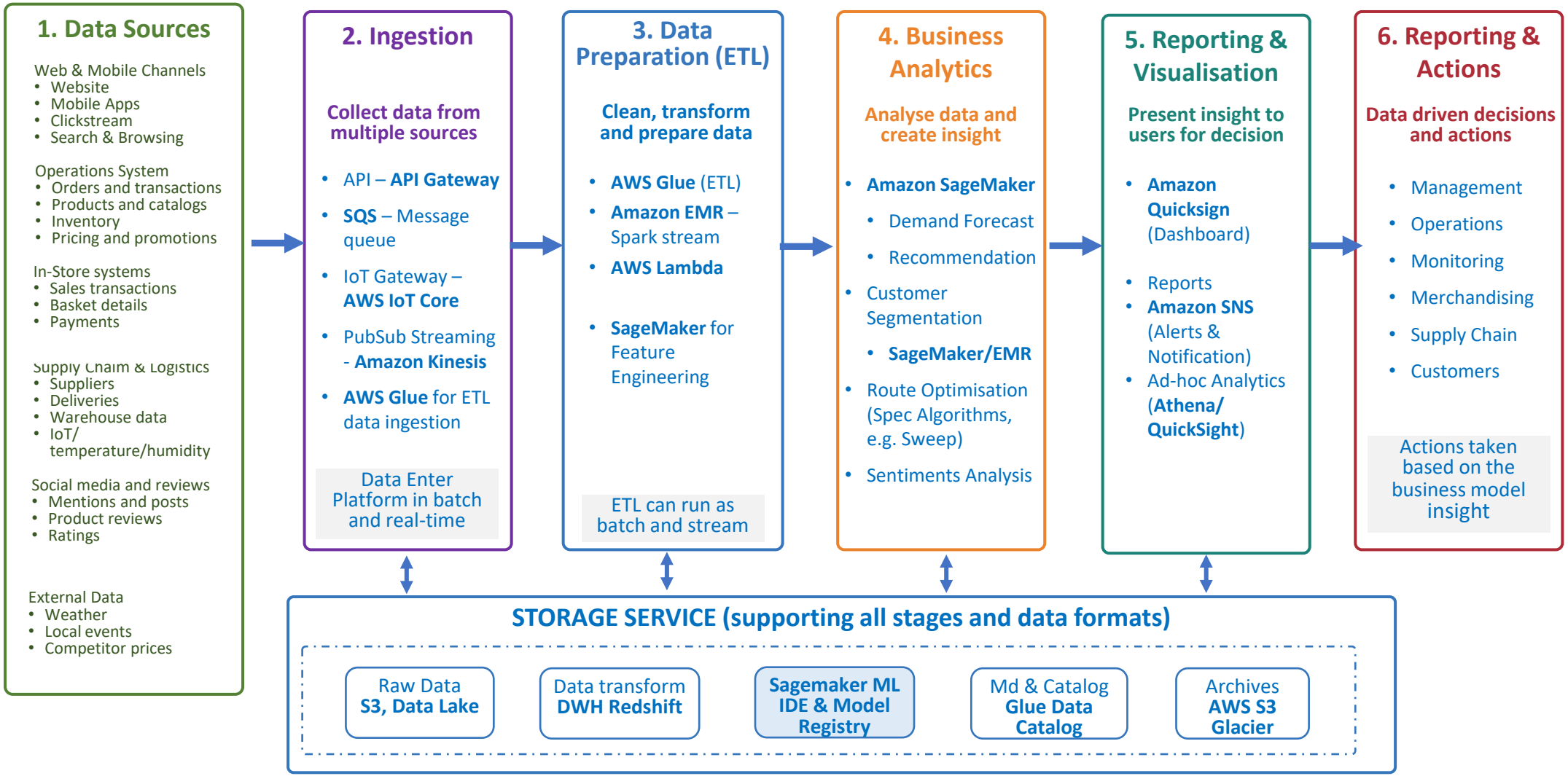
Guiding Examples – Output of Step 4 and Step 5

- Examples – Output of Step 4 and Step 5

FreshRoute From Dataflow to Data Analytics Methods – Summary Results
Step 4 Analytics methods and algorithms selection



FreshRoute From Dataflow to AWS Services – Example Result of Step 5 AWS Cloud and Analytics services selection



Background Information – GenAI and Knowledge Complexity Representation

Knowledge Presentation – Representation - Perception

Available/Received Knowledge	Knowledge inferential/LLM, Internal representation	Perception via Information, Epistemic Validation, Internalisation

Knowledge Presentation

1. Vocabulary/Terminology complexity (KC1)
2. Semantic complexity (KC2)
3. Ontological complexity (KC3)
4. Structural complexity (KC4)
5. Methodological complexity (KC5)
6. Epistemic complexity/validity (KC6)

Knowledge Representation

1. Vocabulary / terminology (K1)
2. Syntax / expression form (K2)
3. Factual / propositional layer (K3)
4. Semantic layer (K4)
5. Ontological layer (K5)
6. Structural / graph layer (K6)
7. Inferential layer (K7)
8. Methodological layer (K8)
9. Epistemic / research-frontier layer (K9)

Knowledge perception

- If a human doesn't have a full Kx or KCx model of KCM, they may treat received/observed information structured wrongly or a kind of imaginary mixed
- This is the case for children
  - Can we control and speed up development?
  - Can we do the same for Doctoral (PhD) Students?
- Children imaginary treat objects and animals with human features
  - Hypothetically, because of their developing perception KCM
  - Agent-centered perception model by children
- TODO: Adding Pragmatics to Semantics Complexity level as related to language understanding by humans

KCM – Knowledge Complexity Model
KCEG – Knowledge Complexity and Epistemic Governance
HGLEP – Human – Governed LLM Epistemic Knowledge Production

Conceptual chain for Human - AI interaction for HGLEP/KCEG



1. Knowledge source
- paper, dataset, prior dialogue, experiment, model, human expertise

2. Knowledge presentation
- text, diagram, table, formula, code, LLM response

3. Knowledge representation
- concepts, relations, ontology, claims, evidence, assumptions, methods

4. Knowledge perception / reconstruction
- human or AI agent interprets and reconstructs the meaning

5. Knowledge assessment
- KCEG evaluates level, completeness, uncertainty, reliability, novelty

6. Internalisation / agent memory update
- human updates personal epistemic model
- AI agent updates context, memory, graph, or retrieval state

7. Knowledge action
- explanation, critique, decision, experiment, paper writing, new prompt

8. Regeneration
- human or AI generates a new presentation

Knowledge is an actional element of the human cognitive composition
- Knowledge structure/direct actions
- Built-in/constructed/re-constructed knowledge structure defines learning efficiency and effective curricula

KCM – Knowledge Complexity Model
KCEG – Knowledge Complexity and Epistemic Governance
HGLEP – Human – Governed LLM Epistemic Knowledge Production

Basis for a generative loop:
presentation → perception → representation → assessment → memory → action → new presentation

Human - AI Epistemic Interaction Model for achieving goal/intention and validated output

Human agent:

Personal epistemic model
- semantic memory
- methodological knowledge
- tacit expertise
- epistemic judgment
- goals / intentions
- **prompt / action**

AI agent:

Agent knowledge state
- **context window (prompt + config context)**
- **retrieved knowledge**
- latent model representation (ML internal)
- symbolic/graph memory
- tool outputs
- generated presentations

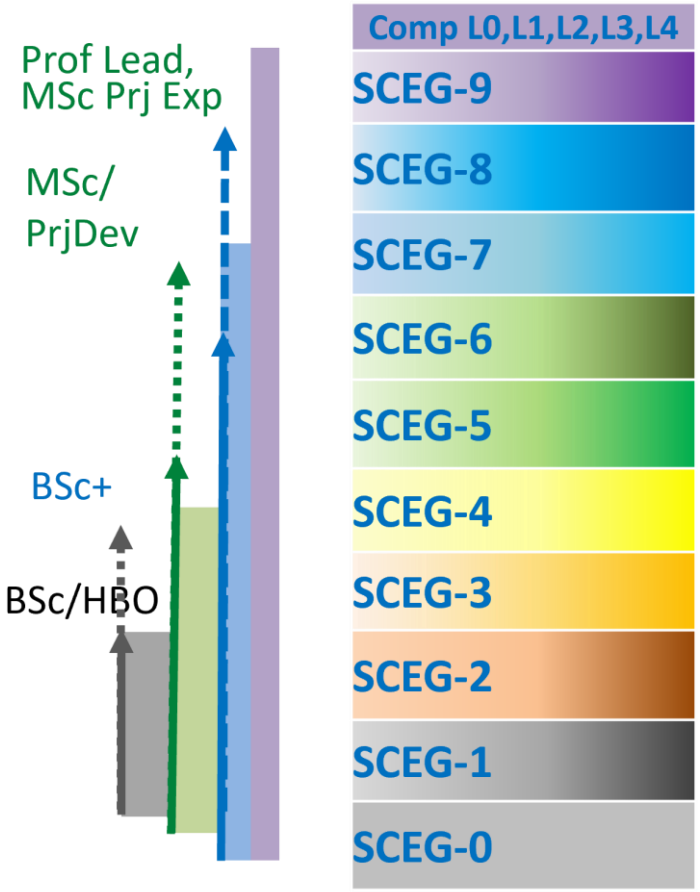
Interaction process:

Human intention
↓
Prompt / instruction presentation
↓
AI internal **representation**
↓
Generated knowledge presentation (output)
↓
Human perception and reconstruction/representation
↓
KCM/KCEG assessment (epistemic judgement by human/researcher)
↓ ↑ ↓ ↑
HGLEP-guided refinement (iterative)
↓
Verified/approved **result/output**
↓
Updated human and AI knowledge states

KCM – Knowledge Complexity Model
KCEG – Knowledge Complexity and Epistemic Governance
HGLEP – Human – Governed LLM Epistemic Knowledge Production

Software Complexity Model (SCEG) as a basis for System and Software Engineering Curriculum with an indication of Competence Levels

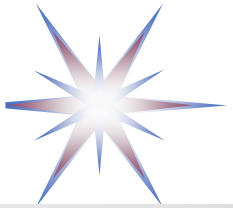
code → modules → data models → APIs → architecture → quality attributes → runtime operation → lifecycle governance



- SCEG-9 Evolution, governance, compliance, sustainability
- SCEG-8 Runtime, deployment, DevOps -> MLOps, CI/CD, observability
- SCEG-7 Quality attributes, verification, security, risk
- SCEG-6 Architecture styles, layers, views, decisions
- SCEG-5 APIs, contracts, data models, semantic interoperability
- SCEG-4 Modules, components, OO/functional design, patterns
- SCEG-3 Functional semantics, requirements traceability
- SCEG-2 Algorithms, data structures, computational logic
- SCEG-1 Code syntax, local implementation, style
- SCEG-0 Prerequisite: Intent, requirements, constraints, acceptance criteria

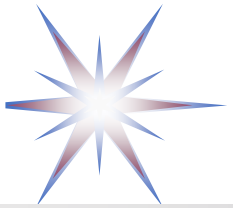
Course design focus
- Sequential courses: SCEG-1 – SCEG-3
- Progressive recurrent topics/courses SCEG-4 – SCEG-8

→ Knowledge/Competence at Level C3+ ⇢ Knowledge/Competence at Level C1+ Awareness



Additional Structural Prompt Engineering (SPE) Examples

- Applying the Foundational Architecture Design Principles and domain/industry related concepts and terms
- Aligning with the Foundational Architecture Design Principles and Contextual Environment
- ENSURE 4M model: Aligning the project 4M - With Foundational Architecture Design Principles



SPE01 - Applying the Foundational Architecture Design Principles and domain/industry related concepts and terms

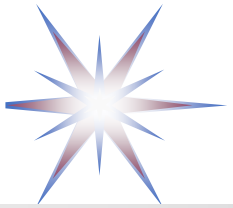
- Applying the Foundational Architecture Design Principles and domain/industry related concepts and terms
  - Uses/Evoke the Model's latent structured knowledge
  - Builds a consistent Personal Skills profile (personal cognitive profile of user)

SPE02 - Aligning with the Foundational Architecture Design Principles and Contextual Environment

- DigitalTwins Environmental Management Application/Service
  - Current implementation is focused on the Carbon Intensity assessment
  - Task is to integrate it with the overarching project (ENSURE)
  - Expectedly compatibility with the GreenDIGIT

Prompt SPE02 Example – Aligning DigitalTwins (initial version) with the project context and FADP

- Step N
  - Digital Twins were, at some stage of digital infrastructure development, treated as a promising technology, in particular for industrial manufacturing systems, also applicable to scientific experimental facilities that require monitoring and control of the facility and object under research.
  - What is the current state of development of Digital Twins? How can an AI agentic system be used (or already used) for Digital Twins? What architectural components can be defined for Digital Twins Agentic Systems?
- Step N+1
  - This is a very valuable proposal; however, it needs to be adjusted to (1) minor specifics in the system design and (2) better aligned with the current EMI architecture.
  - (1) You defined ADTSA as a set of nine principal planes. You defined planes as a set of functionalities and services that correspond to the DT system/infrastructure hierarchical layers. This is what is defined in the system design as layers. Planes are defined as functionalities that operates cross hierarchical layers. So, please redefined ADTSA planes as layers.
  - (2) Attached document describes the current GreenDIGIT EMI layered architecture. Layer ML3 Digital Twins defined as optional, ML3-DT may rely on the layers ML0-ML2 and can report to ML4-ML5. ML6 in GreenDIGIT is defined as User level that hosts the operational dashboard for EMI.
  - In adjusting ADTSA and EMI, I would see the following solution:
    - leave ADTSA architeture layers as currently defined but mention that ADTSA-1 -- ADTSA-3 can be aligned to ML0-ML2 and also use ML4 metrics and KpI services
    - (optionally, may remain indepenent) Layers ADTSA-6 -- ADTSA-8 may interact with ML5
    - ADTSA-9 can be integrated with ML6.
  - So, please this assume linking by showing interlinking between two architectures and not re-defining (at this stage) ADTSA.



• SPE03 - ENSURE 4M model: Aligning with Foundational Architecture Design Principles

- ENSURE 4M model: Measure – Model – Monitor – Manage

- Aligning the project 4M - With Foundational Architecture Design Principles

Recommended books and useful blogs

- Online and blogs
- Books

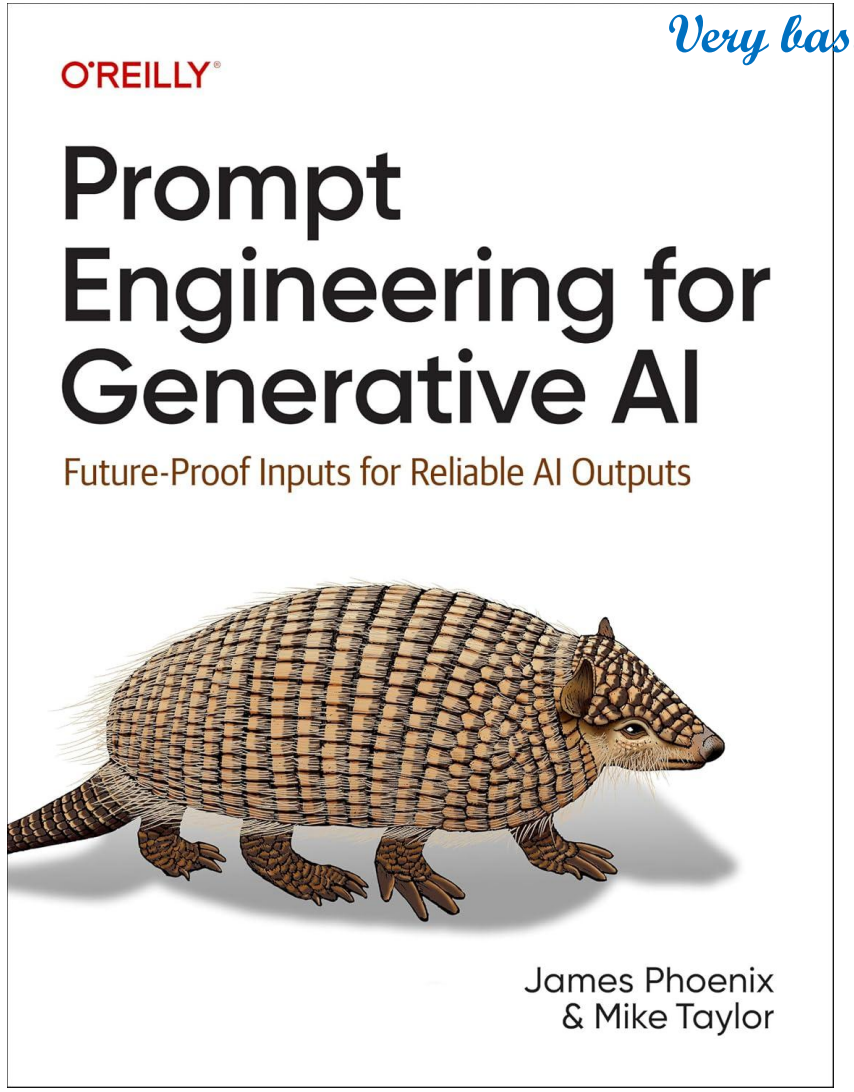
Blogs - Prompt Engineering and Agentic design patterns

- LLMs, RAG, Agents, MCP: The AI Evolution You Must Know (A Visual Explanation): AI Evolved From a Chatbot to a System. Here's the Full Story Nobody Tells You. By Divy Yadav, May 7, 2026
  - https://pub.towardsai.net/llms-rag-agents-mcp-the-ai-evolution-you-must-know-a-visual-explanation-9ee07e421587
- 9 RAG Architectures Every AI Developer Must Know: A Complete Guide with Examples: Architectures beyond Naive Rag to build reliable production AI Systems, by Divy Yadav, Dec 19, 2025
  - https://medium.com/towards-artificial-intelligence/rag-architectures-every-ai-developer-must-know-a-complete-guide-f3524ee68b9c
- Top 20 ChatGPT prompts that every prompt engineers should know, by Vishnu Sivan, May 27, 2023
  - https://medium.com/coinmonks/top-20-chatgpt-prompts-that-every-prompt-engineers-should-know-937b0ea5472
- I Tested 200 ChatGPT Prompts. These Were The Only Ones Worth Saving, by Hazel Paradise, Jul 20, 2026
  - https://medium.com/@hazelparadise/i-tested-200-chatgpt-prompts-these-were-the-only-ones-worth-saving-f0cb9ea57636
- The 15 AI Agent Design Patterns Every Engineer Should Know (With Production Examples) - A field guide for engineers who want the right pattern, not the most impressive one by Aayush Ostwal, May 23, 2026
  - https://ai.plainenglish.io/the-15-ai-agent-patterns-every-engineer-should-know-with-production-examples-9293781e4fdd
- 20 Most Important AI Concepts Explained in Just 20 Minute: Beginner-Friendly Guide, Deep concept, Mar 24, 2026
  - https://medium.com/lets-code-future/20-most-important-ai-concepts-explained-in-just-20-minute-b7dd3ad2b506
- 9 Modern AI Concepts That Finally Made Sense After I Automated My Inbox With Claude: LLM, Agents, MCP, Context engineering, Loop engineering, subagents, Harness engineering, and Human-in-the-loop, explained through one real workflow. By Divy Yadav, June 2026
  - https://medium.com/activated-thinker/i-tested-every-ai-writing-tool-so-you-dont-have-to-only-3-are-worth-it-in-2026-50c602d0cd18
  - https://pub.towardsai.net/9-modern-ai-concepts-that-finally-made-sense-after-i-automated-my-inbox-with-claude-96906b3f5166
- How to Master Data Structures and Algorithms Without Feeling Lost: A beginner-friendly guide to understanding DSA deeply, building real problem-solving skills, and avoiding the surface-level learning that makes you forget everything later. July 2026
  - https://medium.com/lets-code-future/how-to-master-data-structures-and-algorithms-without-feeling-lost-44f85d42b366

Papers – AI-assisted Research & Development

- The Prompt Report: A Systematic Survey of Prompt Engineering Techniques, Feb 2025
  - https://arxiv.org/abs/2406.06608
  - Examples, collection of references
- Empty Shelves or Lost Keys? Recall Is the Bottleneck for Parametric Factuality, Nitay Calderon, Eyal Ben-David, Zorik Gekhman, Eran Ofek, Gal Yona
  - https://arxiv.org/abs/2602.14080
  - Paper demonstrates that factual errors in large language models (LLMs) are rarely due to missing knowledge ("empty shelves"). Instead, they happen because the model fails to retrieve the information it already holds ("lost keys")

Prompt Engineering for Generative AI: Future-Proof Inputs for Reliable AI Outputs at Scale

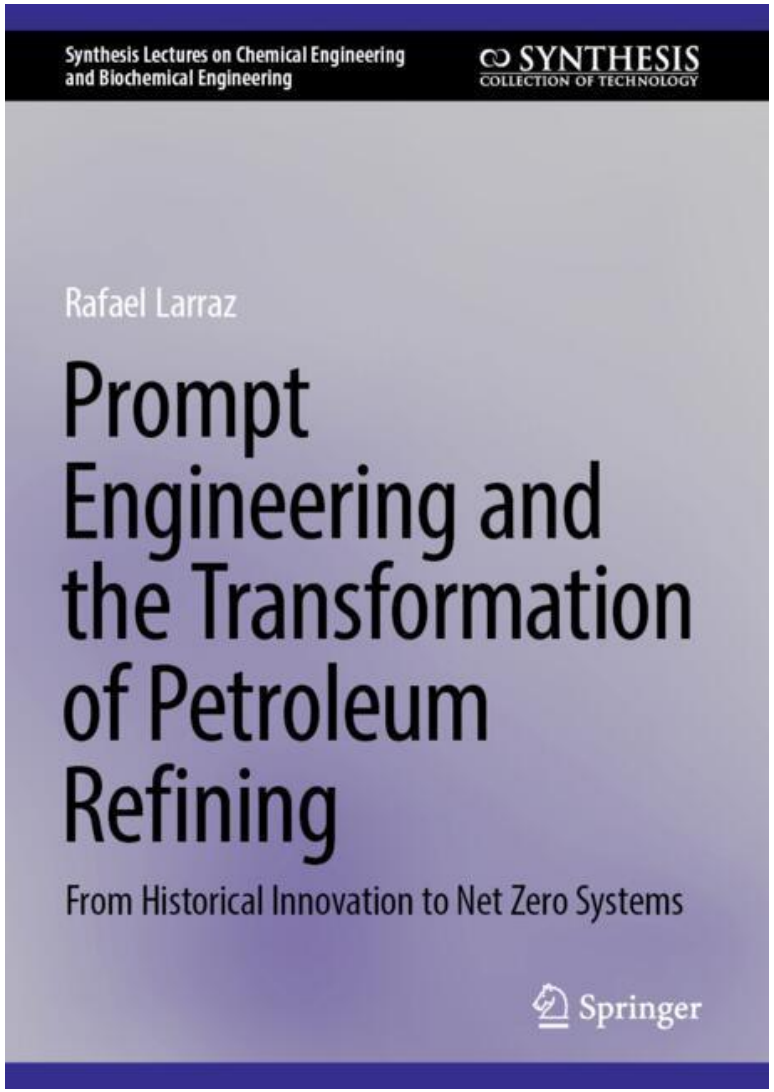


Very basic – Good for entry users

Prompt Engineering for Generative AI: Future-Proof Inputs for Reliable AI Outputs at Scale

- Authors: James Phoenix and Mike Taylor
- Publisher: O'Reilly Media, 2024, 422 pages
- ISBN-13: 978-1098153434
- Focus areas and topics: Prompting Systems, Agent Reasoning Loops, ReAct Patterns, Systematic Evaluation
- Advanced reasoning patterns: Chain-of-Thought (CoT), Tree-of-Thought (ToT), and reasoning loops for agents.
- Structuring behavioral instructions to constrain agent actions, avoid hallucinations, and enforce deterministic output schemas.
- Systematic debugging framework: identifying whether a system failure stems from the base prompt, model limits, or tool execution.
- Transitioning from single-prompt generation to multi-step agent orchestration.

Rafael Larraz, Prompt Engineering and the Transformation of Petroleum Refining: From Historical Innovation to Net Zero Systems

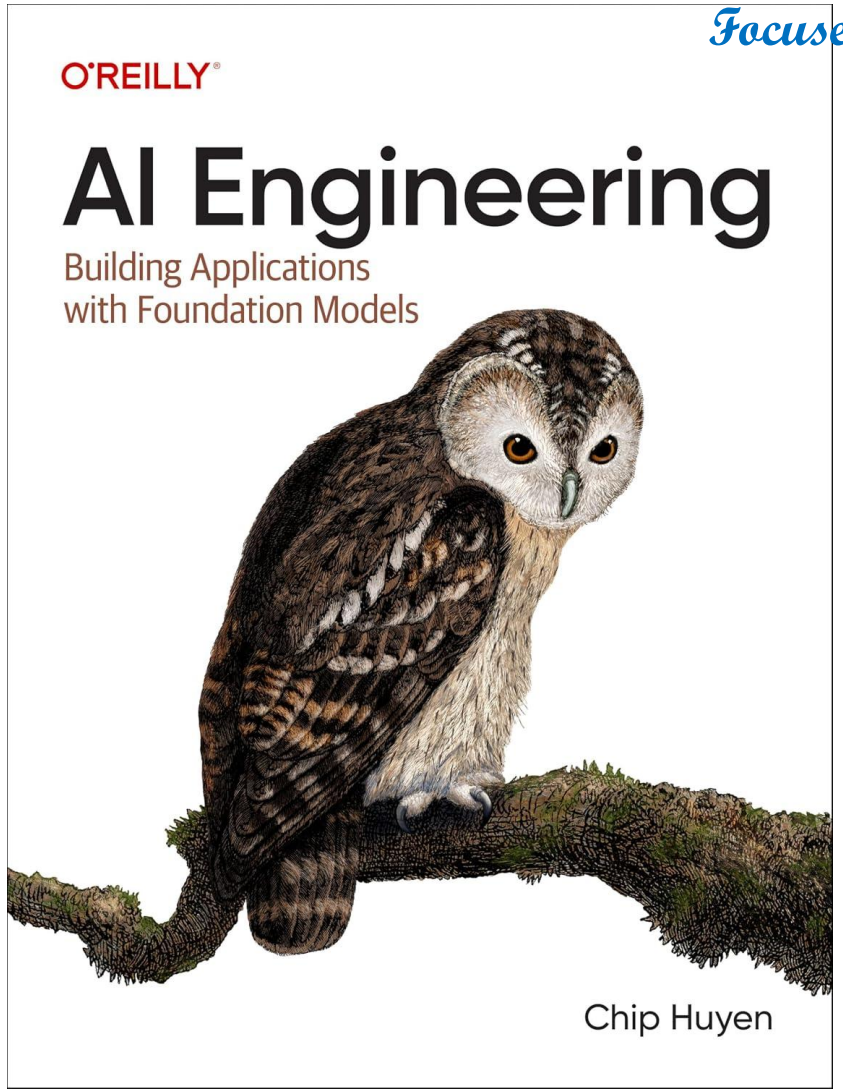


A good example of domain-specific application development that is different from standard software development

Rafael Larraz, Prompt Engineering and the Transformation of Petroleum Refining: From Historical Innovation to Net Zero Systems

- ISBN 978-3-031-99727-3 ISBN 978-3-031-99728-0 (eBook)
  https://doi.org/10.1007/978-3-031-99728-0
  Springer Nature Switzerland AG, 2026
- The book is well-structured with good examples for developing domain-specific applications and addresses architecture, design and prompt engineering aspects for digital systems and infrastructure projects, extending the common software design methods.
- The book has well-described prompt engineering for the domain of Petroleum Refining, which is beyond pure software coding and requires complex architecture and domain knowledge for efficient use of LLMs.

AI Engineering: Building Applications with Foundation Models

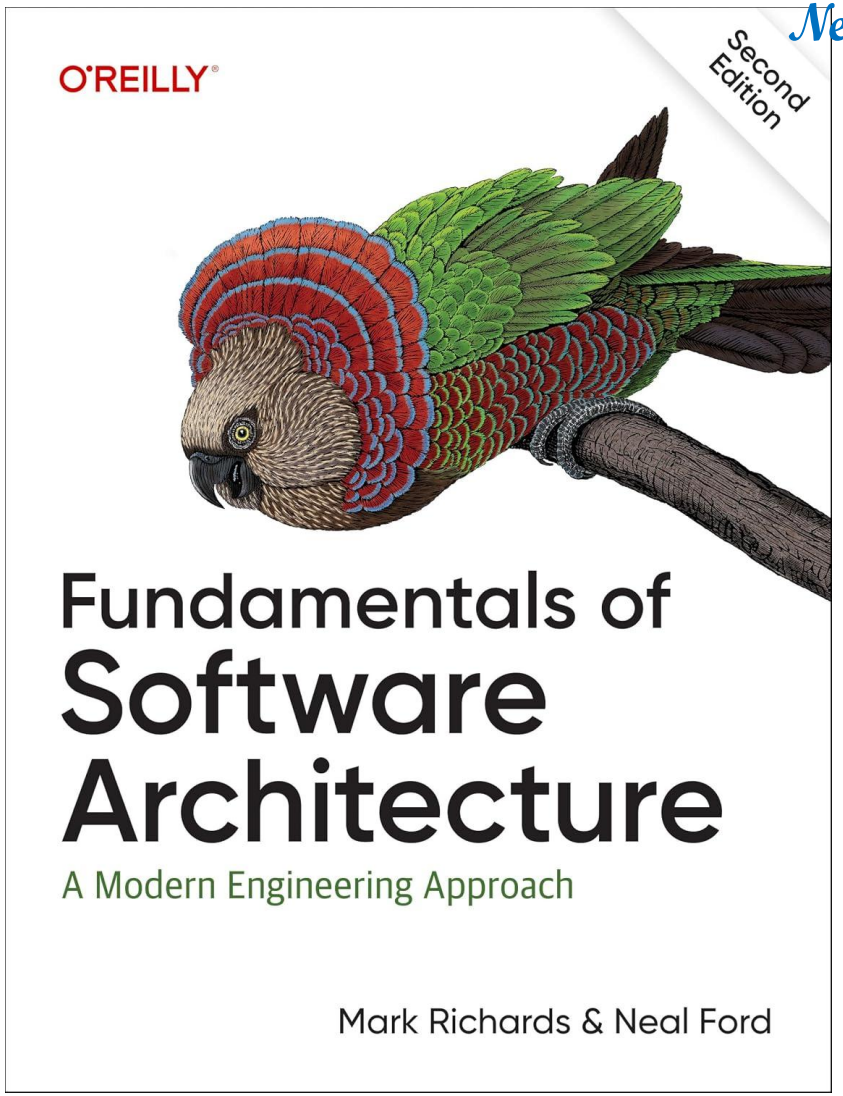


Focused on AI/Agentic systems design

AI Engineering: Building Applications with Foundation Models

- Author: Chip Huyen
- Publisher: O'Reilly Media, 2025, ~450 pages
- ISBN-13: 978-1098166304
- Focus areas and topics: Production Architecture, Non-Deterministic Systems Evaluation, LLMOps, Agent Workflows
- System trade-offs in agentic design: latency vs. accuracy, autonomy vs. human-in-the-loop oversight.
- Architectural patterns for routing, planning, and tool interaction across foundation models.
- Evaluation frameworks and benchmarking strategies for autonomous agent actions and tool use.
- Infrastructure patterns for deploying, monitoring, and debugging complex LLM-driven workflows.
- GitHub version: **AI Engineering Playbook:** Based on *AI Engineering. Building Applications with Foundation Models* by **Chip Huyen** (O'Reilly, 2025) https://github.com/ArnabBir/ai-engineering-explained/tree/main

Fundamentals of Software Architecture, Second edition



Necessary knowledge for advanced projects development and review

Fundamentals of Software Architecture, Second edition

- Authors: Mark Richards, Neal Ford
- Publisher: O'Reilly, 2025, 543 pp.
- ISBN-13 : 978-1098175511
- A comprehensive introduction to architectural thinking, modularity, architectural characteristics, styles, trade-offs, decisions, governance and professional practice.
- The second edition explicitly addresses generative AI in architectural decision-making and its use as an architect's assistant, alongside infrastructure, data and team considerations.